Linux terminal

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Exercises

Exercise 1: Finding Your Way Around

This exercise covers **`pwd`**, **`ls`**, **`cd`**, and basic information commands.

1. Open your terminal. Verify your starting location (your home directory) by printing the working directory.

   ```
   $ pwd
   ```

2. List the contents of your home directory. Then, list them again showing **all** files in the **long** list format.

   ```
   $ ls
   $ ls -la
   ```

3. Navigate to the system log directory at `/var/log` and list its contents.

   ```
   $ cd /var/log
   $ ls
   ```

4. Get some information: find out your username and the current date.

   ```
   $ whoami
   $ date
   ```

5. Return to your home directory using the quickest shortcut.

   ```
   $ cd ~
   ```

Exercise 2: Exploring Key System Directories

Reinforce your knowledge of the filesystem layout by visiting important system directories.

1. Navigate to the `/etc` directory, which holds system-wide configuration files.

   ```
   $ cd /etc
   ```

2. List its contents. You'll see many configuration files.

   ```
   $ ls
   ```

3. View the contents of the `os-release` file to see information about your Linux distribution.

   ```
   $ cat os-release
   ```

4. Now, navigate to the `/bin` directory to see where many essential command programs are stored. List its contents and see if you recognize any.

   ```
   $ cd /bin
   $ ls
   ```

Exercise 3: Creating and Managing Files

In this exercise, you'll create, copy, move, and delete files and directories.

1. From your home directory, create a new directory called `IEI`.

   ```
   $ cd ~
   $ mkdir IEI
   ```

2. Navigate inside your new `IEI` directory.

   ```
   $ cd IEI
   ```

3. Create an empty file called `notes.txt`.

   ```
   $ touch notes.txt
   ```

4. Add text to your file and then view its contents.

   ```
   $ echo "My first line of text." > notes.txt
   $ cat notes.txt
   ```

5. Make a copy of your file named `notes_backup.txt`.

   ```
   $ cp notes.txt notes_backup.txt
   ```

6. Rename `notes.txt` to `important_notes.txt`.

   ```
   $ mv notes.txt important_notes.txt
   ```

7. Clean up by deleting the backup file.

   ```
   $ rm notes_backup.txt
   ```

Exercise 4: Understanding Permissions

This exercise focuses on reading and changing file permissions with **`chmod`**.

1. Inside your `~/IEI` directory, create a new file called `secret_data.txt`.

   ```
   $ touch secret_data.txt
   ```

2. View the file's default permissions.

   ```
   $ ls -l secret_data.txt
   ```

3. Remove all permissions for everyone.

   ```
   $ chmod 000 secret_data.txt
   ```

4. Try to view the file's contents. You should get a **"Permission denied"** error.

   ```
   $ cat secret_data.txt
   ```

5. Restore read and write permission for **only yourself**.

   ```
   $ chmod u+rw secret_data.txt
   ```

6. Create an empty script file `my_script.sh` and make it executable for yourself. Check the permissions afterward to see the change.

   ```
   $ touch my_script.sh
   $ chmod u+x my_script.sh
   $ ls -l my_script.sh
   ```

Exercise 5: Finding Files and Content with `find` and `grep`

Learn to locate files by name and search for text within them.

1. Inside `~/IEI`, create a subdirectory and a new file within it.

   ```
   $ mkdir -p ~/IEI/reports
   $ echo "This is a confidential report." > ~/IEI/reports/report-2025.txt
   ```

2. Use the `find` command to search for any file ending with `.txt` inside your `IEI` directory.

   ```
   $ find ~/IEI -name "*.txt"
   ```

3. Use `grep` to search for the word "confidential" in your new report file. The `-i` flag makes the search case-insensitive.

   ```
   $ grep -i "confidential" ~/IEI/reports/report-2025.txt
   ```

Exercise 6: Managing Processes

Learn how to view and stop running programs from the command line.

1. Start a process that will run in the background. The `sleep` command waits for a specified number of seconds, and the & sends it to the background.

   ```
   $ sleep 120 &
   ```

2. Find the Process ID (PID) of the `sleep` command. You can use `pgrep` for this.

   ```
   $ pgrep sleep
   ```

3. Now, terminate the process using the `kill` command and the PID you just found. Replace `PID` with the actual number from the previous step.

   ```
   $ kill PID
   ```

4. Verify that the process is no longer running. The `pgrep sleep` command should now return nothing.

   ```
   $ pgrep sleep
   ```

Exercise 7: Managing Software with APT

Let's install and remove a program using the **APT** package manager.

1. First, synchronize your system's package list with the software repositories.

   ```
   $ sudo apt update
   ```

2. Search for a useful command-line tool called `htop`.

   ```
   $ apt search htop
   ```

3. Now, install `htop`. You will need to confirm the installation when prompted.

   ```
   $ sudo apt install htop
   ```

4. Run the program you just installed. Press q to quit.

   ```
   $ htop
   ```

5. Finally, clean up by removing the package from your system.

   ```
   $ sudo apt remove htop
   ```

Exercise 8: Combining Commands

Let's explore the power of the **pipe (|)** and **redirection (>>)**.

1. The command `ps aux` lists all running processes. Use the pipe (`|`) to send this output to `grep` to find your own "bash" process.

   ```
   $ ps aux | grep "bash"
   ```

2. Create a log file with one entry.

   ```
   $ echo "$(date): Starting my work." > ~/IEI/activity.log
   ```

3. Use the append operator (`>>`) to add a second line to the file without deleting the first one.

   ```
   $ echo "$(date): Finished exercise 8." >> ~/IEI/activity.log
   ```

4. Verify that your log file contains both lines.

   ```
   $ cat ~/IEI/activity.log
   ```

Exercise 9: Customizing Your Environment

Time to edit your **`.bashrc`** file to create a handy shortcut (an alias).

1. Open your `~/.bashrc` file using the nano editor.

   ```
   $ nano ~/.bashrc
   ```

2. Scroll to the very bottom and add the following line to create a shortcut `ll` for the command `ls -alF`.

   ```
   alias ll='ls -alF'
   ```

3. Save the file and exit nano (`Ctrl+X`, then `Y`, then `Enter`).

4. Load the changes into your current session.

   ```
   $ source ~/.bashrc
   ```

5. Test your new alias.

   ```
   $ ll
   ```

Exercise 10: Understanding the $PATH Variable

Discover how the shell finds the commands you run.

1. View the current `$PATH` variable. It's a colon-separated list of directories.

   ```
   $ echo $PATH
   ```

2. Create a simple one-line script in your `~/IEI` directory and make it executable.

   ```
   $ echo '#!/bin/bash' > ~/IEI/hello
   $ echo 'echo "Hello from my custom script!"' >> ~/IEI/hello
   $ chmod +x ~/IEI/hello
   ```

3. Try to run the script by name. It will fail because it's not in a directory listed in `$PATH`.

   ```
   $ hello
   ```

4. Now run it using its relative path. This works.

   ```
   $ ./hello
   ```

5. Temporarily add your `~/IEI` directory to the `$PATH`. Now try running the script by name again.

   ```
   $ export PATH="$HOME/IEI:$PATH"
   $ hello
   ```

This change only lasts for your current terminal session.

Exercise 11: Scripting Challenge

Let's create a script that automates creating a project structure.

1. Create and open a new file named `setup_project.sh` in your `~/IEI` directory. Add the following code, then save and close the file.

```
#!/bin/bash
PROJECT_DIR="$HOME/IEI/my_project"

if [ -d "$PROJECT_DIR" ]; then
  echo "Error: Directory '$PROJECT_DIR' already exists."
  exit 1
fi

mkdir "$PROJECT_DIR"
echo "Directory '$PROJECT_DIR' created."

for folder in assets source docs
do
  mkdir "$PROJECT_DIR/$folder"
  echo "-> Created subfolder: $folder"
done

echo "Project setup complete!"
```

2. Make the script executable and then run it.

```
$ chmod +x ~/IEI/setup_project.sh
$ ~/IEI/setup_project.sh
```

3. Verify that the directory and its subdirectories were created.

```
$ ls -R ~/IEI/my_project
```

Exercise 12: Scheduling a Task with `cron`

Let's create a simple script and schedule it to run automatically every minute.

1. **Create the Script:** In your `~/IEI` directory, create a script named `log_time.sh` with the following content.

```
#!/bin/bash
date >> $HOME/IEI/cron_log.txt
```

2. **Make it Executable:**

```
$ chmod +x ~/IEI/log_time.sh
```

3. **Open your Crontab:** This will open a text editor.

```
$ crontab -e
```

4. **Add the Cron Job:** Go to the bottom of the file and add the following line. You must use the full, absolute path to your script.

```
* * * * * /home/student/IEI/log_time.sh
```

5. **Save and Verify:** Save and exit the editor. Wait two minutes, then check your log file. You should see two timestamp entries.

```
$ cat ~/IEI/cron_log.txt
```

6. **Clean Up:** It's very important to remove the cron job so it doesn't run forever. This command removes your entire crontab file.

   ```
   $ crontab -r
   ```

Exercise 13: Viewing Files with `cat`, `less`, head, and `tail`

Practice the different ways to inspect file contents without opening an editor.

1. Start by creating a file with several lines so you have something to work with.

   ```
   $ seq 1 100 > ~/IEI/numbers.txt
   ```

2. Use `cat` to dump the entire file to the screen. Notice how it scrolls past quickly.

   ```
   $ cat ~/IEI/numbers.txt
   ```

3. Now use `less` to open the same file in a scrollable viewer. Use the **Arrow Keys** to scroll and press **q** to quit.

   ```
   $ less ~/IEI/numbers.txt
   ```

4. View only the **first 5 lines** of the file using head.

   ```
   $ head -n 5 ~/IEI/numbers.txt
   ```

5. View only the **last 5 lines** of the file using `tail`.

   ```
   $ tail -n 5 ~/IEI/numbers.txt
   ```

6. Combine head and `tail` with a pipe to extract **only lines 45 through 55** from the file.

   ```
   $ head -n 55 ~/IEI/numbers.txt | tail -n 11
   ```

7. Use wc (word count) to count the total number of lines, words, and characters in the file.

   ```
   $ wc ~/IEI/numbers.txt
   ```

Exercise 14: System Information Deep-Dive

Use the terminal to gather detailed information about your system's hardware and resources.

1. Display your kernel version and system architecture.

   ```
   $ uname -a
   ```

2. Check your CPU information by reading from the virtual `/proc` filesystem. Filter the output to show only the model name.

   ```
   $ cat /proc/cpuinfo | grep "model name"
   ```

3. Display your current RAM and Swap usage in a human-readable format.

   ```
   $ free -h
   ```

4. Check disk space usage across all mounted filesystems.

   ```
   $ df -h
   ```

5. Check disk usage of your home directory specifically. The `-s` flag gives a summary and `-h` makes it human-readable.

   ```
   $ du -sh ~
   ```

6. If you have sudo access, use `dmidecode` to query the system's BIOS information.

   ```
   $ sudo dmidecode -t bios
   ```

7. View your network interfaces and their IP addresses.

   ```
   $ ip addr show
   ```

Exercise 15: Wildcards and Globbing

Learn to select multiple files at once using pattern matching.

1. Create a set of test files to work with inside a new directory.

   ```
   $ mkdir -p ~/IEI/wildcard_test
   $ cd ~/IEI/wildcard_test
   $ touch report1.txt report2.txt report3.txt
   $ touch summary.txt data.csv output.csv
   $ touch image1.png image2.png image10.png
   ```

2. Use the * wildcard to list **all `.txt` files**.

   ```
   $ ls *.txt
   ```

3. Use the * wildcard to list **all files starting with `report`**.

   ```
   $ ls report*
   ```

4. Use the ? wildcard to match files with **exactly one character** after `report`. Notice that `report10.txt` is **not** matched.

   ```
   $ ls report?.txt
   ```

5. Use the ? wildcard to match image files with a **single-digit** number. Notice that `image10.png` is excluded.

   ```
   $ ls image?.png
   ```

6. List **all `.csv` files** and redirect the output to a file called `csv_list.txt`.

   ```
   $ ls *.csv > csv_list.txt
   $ cat csv_list.txt
   ```

7. Use wildcards to **delete all `.csv` files** at once, then verify they are gone.

   ```
   $ rm *.csv
   $ ls
   ```

8. Clean up the test directory.

   ```
   $ cd ~
   $ rm -r ~/IEI/wildcard_test
   ```

Exercise 16: I/O Streams and Error Redirection

Understand how to control where standard output and standard error go.

1. Run a command that produces **normal output** (stdout). Redirect it to a file.

   ```
   $ echo "This is standard output" > ~/IEI/stdout_test.txt
   $ cat ~/IEI/stdout_test.txt
   ```

2. Run a command that will produce an **error** (stderr). Try to list a directory that doesn't exist.

   ```
   $ ls /nonexistent_directory
   ```

3. Redirect **only the error** to a file using 2>. The error message will go to the file instead of the screen.

   ```
   $ ls /nonexistent_directory 2> ~/IEI/errors.log
   $ cat ~/IEI/errors.log
   ```

4. Now run a command that produces **both** stdout and stderr. Use `find` to search `/etc` — some directories will produce "Permission denied" errors.

   ```
   $ find /etc -name "*.conf"
   ```

5. Separate the two streams: save **results** to one file and **errors** to another.

```
$ find /etc -name "*.conf" > ~/IEI/results.txt 2> ~/IEI/find_errors.log
$ wc -l ~/IEI/results.txt
$ cat ~/IEI/find_errors.log
```

6. Combine **both streams** into a single file using 2>&1.

```
$ find /etc -name "*.conf" > ~/IEI/all_output.txt 2>&1
$ wc -l ~/IEI/all_output.txt
```

7. Discard all output entirely by redirecting to `/dev/null` (the system's "black hole").

```
$ find /etc -name "*.conf" > /dev/null 2>&1
```

Exercise 17: User Management

Practice creating, modifying, and removing user accounts. These commands require sudo.

1. Create a new user called `testuser`.

```
$ sudo useradd testuser
```

2. Verify the user was created by checking the `/etc/passwd` file.

```
$ grep testuser /etc/passwd
```

3. Set a password for the new user. You will be prompted to type the password twice.

```
$ sudo passwd testuser
```

4. Check what groups the new user belongs to.

```
$ groups testuser
```

5. Add the user to the `sudo` group so they have administrator privileges.

```
$ sudo usermod -aG sudo testuser
$ groups testuser
```

6. Switch to the new user's account temporarily using `su`. Type `exit` to return to your own account.

```
$ su - testuser
$ whoami
$ pwd
$ exit
```

7. Delete the user and their home directory to clean up.

```
$ sudo userdel -r testuser
$ grep testuser /etc/passwd
```

Exercise 18: Numeric (Octal) Permissions with `chmod`

Learn to use the numeric notation for setting permissions, which is faster than symbolic notation for complex changes.

The permission values are: **Read (4)**, **Write (2)**, **Execute (1)**. Add them together for each group: **Owner | Group | Others**.

1. Create a test file and a test script inside `~/IEI`.

```
$ echo "Sensitive data" > ~/IEI/config_file.txt
$ echo '#!/bin/bash' > ~/IEI/run_me.sh
$ echo 'echo "Script executed!"' >> ~/IEI/run_me.sh
```

2. View the current permissions of both files.

```
$ ls -l ~/IEI/config_file.txt ~/IEI/run_me.sh
```

3. Set `config_file.txt` to permission **644** (owner: read+write, group: read, others: read). This is the standard permission for configuration files.

   ```
   $ chmod 644 ~/IEI/config_file.txt
   $ ls -l ~/IEI/config_file.txt
   ```

4. Set `run_me.sh` to permission **755** (owner: read+write+execute, group: read+execute, others: read+execute). This is the standard permission for scripts.

   ```
   $ chmod 755 ~/IEI/run_me.sh
   $ ls -l ~/IEI/run_me.sh
   ```

5. Run the script to confirm it works.

   ```
   $ ~/IEI/run_me.sh
   ```

6. Set `config_file.txt` to **600** (owner: read+write, everyone else: nothing). This is appropriate for private files like SSH keys.

   ```
   $ chmod 600 ~/IEI/config_file.txt
   $ ls -l ~/IEI/config_file.txt
   ```

7. **Challenge:** What permission number would give the owner full access, the group read-only access, and others no access at all? Set it on `run_me.sh` and verify.

   ```
   $ chmod 740 ~/IEI/run_me.sh
   $ ls -l ~/IEI/run_me.sh
   ```